

13.5_muestra36_final

May 4, 2026

```
[21]: from pathlib import Path
import pandas as pd
import numpy as np
```

```
[22]: # =====
# 1) rutas
# =====
PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")

TEST_SFT_CSV = PROJECT_ROOT / "data" / "sft_ready" / "feina_repr30_test_sft.csv"
BASE_REPR_CSV = PROJECT_ROOT / "data" / "splits" / "feina_repr30_test.csv"

OUT_DIR = PROJECT_ROOT / "outputs" / "final_sample_36"
OUT_DIR.mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42
TARGET_N = 36

print("TEST_SFT_CSV :", TEST_SFT_CSV, TEST_SFT_CSV.exists())
print("BASE_REPR_CSV:", BASE_REPR_CSV, BASE_REPR_CSV.exists())
print("OUT_DIR      :", OUT_DIR)
```

```
TEST_SFT_CSV : /home/harielpadillasanchez/Documentos/TT/TT2/data/sft_ready/feina
_repr30_test_sft.csv True
BASE_REPR_CSV:
/home/harielpadillasanchez/Documentos/TT/TT2/data/splits/feina_repr30_test.csv
True
OUT_DIR      :
/home/harielpadillasanchez/Documentos/TT/TT2/outputs/final_sample_36
```

```
[23]: # =====
# 2) cargar archivos
# =====
df_test_sft = pd.read_csv(TEST_SFT_CSV)
df_base = pd.read_csv(BASE_REPR_CSV)

print("\nShape df_test_sft:", df_test_sft.shape)
print("Shape df_base      :", df_base.shape)
```

```
display(df_test_sft.head(3))
display(df_base.head(3))

print("\nColumnas test_sft:")
print(df_test_sft.columns.tolist())

print("\nColumnas base:")
print(df_base.columns.tolist())
```

```
Shape df_test_sft: (238, 4)
Shape df_base      : (238, 32)
```

	row_id	instruction \
0	92	Reescribe en español el siguiente texto con le...
1	115	Reescribe en español el siguiente texto con le...
2	118	Reescribe en español el siguiente texto con le...

	output \
0	Muchas personas no tienen el hábito de presupu...
1	Uno de los mayores problemas de las personas, ...
2	El comportamiento del ingreso marca etapas o c...

	text
0	Reescribe en español el siguiente texto con le...
1	Reescribe en español el siguiente texto con le...
2	Reescribe en español el siguiente texto con le...

	row_id	idFinal \
0	92	7617b_LibroBAC.pdf
1	115	7677_LibroBAC.pdf
2	118	7683_LibroBAC.pdf

	source_text \
0	Muchas personas, más de las que uno pueda imag...
1	Uno de los mayores problemas que enfrentan las...
2	El comportamiento del ingreso marca etapas o c...

	reference_text	idcod	atr0	atr1 \
0	Muchas personas no tienen el hábito de presupu...	Fiorella	15.0	16.0
1	Uno de los mayores problemas de las personas, ...	Fiorella	15.0	17.0
2	El comportamiento del ingreso marca etapas o c...	Fiorella	15.0	14.0

	atr2	atr3	atr4	...	src_words	src_sentences	has_number	has_money \
0	5.0	NaN	NaN	...	35	1	0	0
1	5.0	6.0	4.0	...	50	1	0	0
2	5.0	NaN	NaN	...	49	1	1	0

	has_percent	n_rules_bucket	length_bucket	lex_bucket	\
0	0	r3	long	0	
1	0	r4plus	long	0	
2	0	r3	long	0	

	stratum \				
0	fam=structural		rules=r3		len=long num=0 ...
1	fam=structural		rules=r4plus		len=long num...
2	fam=structural		rules=r3		len=long num=1 ...

	split_stratum				
0	fam=structural		rules=r3		len=long
1	fam=structural		rules=r4plus		len=long
2	fam=structural		rules=r3		len=long

[3 rows x 32 columns]

Columnas test_sft:

['row_id', 'instruction', 'output', 'text']

Columnas base:

['row_id', 'idFinal', 'source_text', 'reference_text', 'idcod', 'atr0', 'atr1', 'atr2', 'atr3', 'atr4', 'atr5', 'atr6', 'atr7', 'atr8', 'lex', 'rules_list', 'n_rules', 'main_rule', 'main_rule_name', 'families_list', 'n_families', 'main_family', 'src_words', 'src_sentences', 'has_number', 'has_money', 'has_percent', 'n_rules_bucket', 'length_bucket', 'lex_bucket', 'stratum', 'split_stratum']

```
[24]: # =====
# 3) detectar columnas
# =====
test_id_col = "row_id" if "row_id" in df_test_sft.columns else ("id" if "id" in
↳df_test_sft.columns else None)
base_id_col = "row_id" if "row_id" in df_base.columns else ("id" if "id" in
↳df_base.columns else None)

if test_id_col is None or base_id_col is None:
    raise ValueError("No encuentre columnas id/row_id para hacer el cruce.")

text_col = "source_text" if "source_text" in df_base.columns else ("Segmento"
↳if "Segmento" in df_base.columns else None)

if text_col is None:
    raise ValueError("No encuentre columna de texto base (source_text o
↳Segmento).")

print("\ntest_id_col:", test_id_col)
```

```
print("base_id_col:", base_id_col)
print("text_col   :", text_col)
```

```
test_id_col: row_id
base_id_col: row_id
text_col    : source_text
```

```
[25]: # =====
# 4) quedarnos solo con los 238 del test
# =====
test_ids = set(df_test_sft[test_id_col].tolist())

df_test_base = (
    df_base[df_base[base_id_col].isin(test_ids)]
    .copy()
    .reset_index(drop=True)
)

print("\nShape df_test_base:", df_test_base.shape)
print("IDs unicos en df_test_base:", df_test_base[base_id_col].nunique())

if df_test_base[base_id_col].nunique() != len(test_ids):
    faltan = sorted(list(test_ids - set(df_test_base[base_id_col].tolist())))
    print("WARNING: faltan ids al cruzar:", len(faltan))
    print("Primeros faltantes:", faltan[:10])

display(df_test_base.head(3))
```

```
Shape df_test_base: (238, 32)
```

```
IDs unicos en df_test_base: 238
```

```
   row_id   idFinal \
0      92  7617b_LibroBAC.pdf
1     115  7677_LibroBAC.pdf
2     118  7683_LibroBAC.pdf
```

```
   source_text \
0  Muchas personas, más de las que uno pueda imag...
1  Uno de los mayores problemas que enfrentan las...
2  El comportamiento del ingreso marca etapas o c...
```

```
   reference_text   idcod  atr0  atr1 \
0  Muchas personas no tienen el hábito de presupu...  Fiorella  15.0  16.0
1  Uno de los mayores problemas de las personas, ...  Fiorella  15.0  17.0
2  El comportamiento del ingreso marca etapas o c...  Fiorella  15.0  14.0
```

```
   atr2  atr3  atr4  ...  src_words  src_sentences  has_number  has_money \
```

0	5.0	NaN	NaN	...	35	1	0	0
1	5.0	6.0	4.0	...	50	1	0	0
2	5.0	NaN	NaN	...	49	1	1	0

	has_percent	n_rules_bucket	length_bucket	lex_bucket	\
0	0	r3	long	0	
1	0	r4plus	long	0	
2	0	r3	long	0	

	stratum \			
0	fam=structural	rules=r3	len=long	num=0 ...
1	fam=structural	rules=r4plus	len=long	num=...
2	fam=structural	rules=r3	len=long	num=1 ...

	split_stratum
0	fam=structural rules=r3 len=long
1	fam=structural rules=r4plus len=long
2	fam=structural rules=r3 len=long

[3 rows x 32 columns]

```
[26]: # =====
# 5) usar estrato general ya existente
# =====
df_feat = df_test_base.copy()

if "split_stratum" not in df_feat.columns:
    raise ValueError("No existe la columna split_stratum en df_test_base")

STRATUM_COL = "split_stratum"

print("\nUsando columna de estratificacion:", STRATUM_COL)
print("Numero de estratos:", df_feat[STRATUM_COL].nunique())

display(
    df_feat[STRATUM_COL]
    .value_counts()
    .rename_axis(STRATUM_COL)
    .reset_index(name="count")
    .head(20)
)
```

Usando columna de estratificacion: split_stratum
Numero de estratos: 39

	split_stratum	count
0	fam=reduction rules=r2 len=medium	20
1	fam=structural rules=r4plus len=long	16

2	fam=structural rules=r2 len=long	16
3	fam=reduction rules=r1 len=medium	15
4	fam=structural rules=r3 len=long	13
5	fam=reduction rules=r2 len=long	12
6	fam=structural rules=r2 len=medium	12
7	fam=lexical rules=r2 len=medium	10
8	fam=reduction rules=r4plus len=long	9
9	fam=lexical rules=r3 len=medium	8
10	fam=reduction rules=r1 len=long	8
11	fam=structural rules=r1 len=medium	8
12	fam=lexical rules=r1 len=short	8
13	fam=lexical rules=r1 len=medium	7
14	fam=reduction rules=r3 len=medium	7
15	fam=reduction rules=r3 len=long	7
16	fam=reduction rules=r1 len=short	6
17	fam=structural rules=r3 len=medium	5
18	fam=lexical rules=r4plus len=long	5
19	fam=lexical rules=r3 len=long	5

```
[27]: # =====
# 6) cuotas proporcionales
# =====
group_counts = (
    df_feat.groupby(STRATUM_COL, as_index=False)
    .size()
    .rename(columns={"size": "n_group"})
)

group_counts["raw_quota"] = group_counts["n_group"] / len(df_feat) * TARGET_N
group_counts["quota"] = np.floor(group_counts["raw_quota"]).astype(int)

# minimo 1 por estrato general
group_counts.loc[group_counts["quota"] == 0, "quota"] = 1

current_total = group_counts["quota"].sum()

if current_total > TARGET_N:
    excess = current_total - TARGET_N
    removable = group_counts[group_counts["quota"] > 1].copy()
    removable["frac"] = removable["raw_quota"] - np.
    ↪ floor(removable["raw_quota"])
    removable = removable.sort_values(["frac", "n_group"], ascending=[True, ↪
    ↪ True])

    idxs = removable.index.tolist()
    i = 0
    while excess > 0 and i < len(idxs):
```

```

        idx = idxs[i]
        if group_counts.loc[idx, "quota"] > 1:
            group_counts.loc[idx, "quota"] -= 1
            excess -= 1
        i += 1

elif current_total < TARGET_N:
    deficit = TARGET_N - current_total
    group_counts["frac"] = group_counts["raw_quota"] - np.
    ↪floor(group_counts["raw_quota"])
    group_counts = group_counts.sort_values(["frac", "n_group"],
    ↪ascending=[False, False])

    idxs = group_counts.index.tolist()
    i = 0
    while deficit > 0:
        idx = idxs[i % len(idxs)]
        if group_counts.loc[idx, "quota"] < group_counts.loc[idx, "n_group"]:
            group_counts.loc[idx, "quota"] += 1
            deficit -= 1
        i += 1

group_counts = group_counts.sort_values(STRATUM_COL).reset_index(drop=True)

print("\nTotal final cuotas:", group_counts["quota"].sum())
display(group_counts.head(20))

```

Total final cuotas: 40

	split_stratum	n_group	raw_quota	quota
0	fam=lexical rules=r1 len=long	1	0.151261	1
1	fam=lexical rules=r1 len=medium	7	1.058824	1
2	fam=lexical rules=r1 len=short	8	1.210084	1
3	fam=lexical rules=r2 len=long	3	0.453782	1
4	fam=lexical rules=r2 len=medium	10	1.512605	1
5	fam=lexical rules=r2 len=short	5	0.756303	1
6	fam=lexical rules=r3 len=long	5	0.756303	1
7	fam=lexical rules=r3 len=medium	8	1.210084	1
8	fam=lexical rules=r3 len=short	1	0.151261	1
9	fam=lexical rules=r4plus len=long	5	0.756303	1
10	fam=lexical rules=r4plus len=medium	1	0.151261	1
11	fam=morphosyntactic rules=r1 len=long	1	0.151261	1
12	fam=morphosyntactic rules=r1 len=medium	1	0.151261	1
13	fam=morphosyntactic rules=r2 len=medium	2	0.302521	1
14	fam=morphosyntactic rules=r2 len=short	1	0.151261	1
15	fam=morphosyntactic rules=r3 len=long	1	0.151261	1
16	fam=morphosyntactic rules=r3 len=medium	4	0.605042	1

17	fam=morphosyntactic rules=r4plus len=long	1	0.151261	1
18	fam=reduction rules=r1 len=long	8	1.210084	1
19	fam=reduction rules=r1 len=medium	15	2.268908	1

```
[28]: # =====
# 7) muestreo reproducible por split_stratum
# =====
rng = np.random.default_rng(RANDOM_STATE)
sample_parts = []

for _, g in group_counts.iterrows():
    stratum_value = g[STRATUM_COL]
    quota = int(g["quota"])

    df_g = df_feat[df_feat[STRATUM_COL] == stratum_value].copy()

    if quota >= len(df_g):
        sample_parts.append(df_g)
    else:
        sampled_idx = rng.choice(df_g.index.to_numpy(), size=quota,
↪replace=False)
        sample_parts.append(df_g.loc[sampled_idx].copy())

sample36_df = pd.concat(sample_parts, ignore_index=True)

# ajuste de seguridad
if len(sample36_df) > TARGET_N:
    sample36_df = sample36_df.sample(TARGET_N, random_state=RANDOM_STATE).copy()
elif len(sample36_df) < TARGET_N:
    faltan = TARGET_N - len(sample36_df)
    restantes = df_feat.loc[~df_feat[base_id_col].
↪isin(sample36_df[base_id_col])].copy()
    extra = restantes.sample(faltan, random_state=RANDOM_STATE)
    sample36_df = pd.concat([sample36_df, extra], ignore_index=True)

sample36_df = sample36_df.sort_values(base_id_col).reset_index(drop=True)

print("\nShape final sample36:", sample36_df.shape)
display(
    sample36_df[
        [base_id_col, text_col, STRATUM_COL]
    ].head(15)
)
```

Shape final sample36: (36, 32)

row_id

source_text \


```

0      550  No tiene incentivos ni motivación, dos element...
1      571  En los procesos de importaciones y exportacion...
2      885  El robo de identidad ocurre cuando su informac...
3     1193  Los sueños de los niños tienen una gran varied...
4     1391  Expresiones como soborno y cohecho se refieren...
5     1399  DIRECCIÓN DE PRESUPUESTO (DIPRES): "formula y ...
6     1636  Interés compuesto: Es la suma del capital y el...
7     1740  La producción en un país se realiza mediante l...
8     1742  A pesar de que algunos critican al intermediar...
9     1790  El impuesto a los réditos es también un buen i...
10     1920  También es de mucha ayuda separar las metas en...
11     2027  Seguridad financiera: Consiste en cumplir con ...
12     2065  La última, pero no menos importante razón, pue...
13     2365  Si el entusiasmo, y esto no es idealismo rampl...
14     2449  En la segunda etapa de programación, se define...

```

```

                                split_stratum
0      fam=reduction | rules=r2 | len=short
1      fam=structural | rules=r4plus | len=long
2      fam=morphosyntactic | rules=r1 | len=long
3      fam=morphosyntactic | rules=r2 | len=medium
4      fam=lexical | rules=r4plus | len=medium
5      fam=reduction | rules=r3 | len=medium
6      fam=reduction | rules=r4plus | len=long
7      fam=morphosyntactic | rules=r3 | len=long
8      fam=morphosyntactic | rules=r3 | len=medium
9      fam=lexical | rules=r2 | len=medium
10     fam=reduction | rules=r3 | len=long
11     fam=structural | rules=r3 | len=long
12     fam=reduction | rules=r2 | len=medium
13     fam=reduction | rules=r2 | len=long
14     fam=structural | rules=r2 | len=long

```

```

[29]: # =====
# 8) guardar
# =====
sample36_csv = OUT_DIR / "feina_repr30_test_sample36_representative.csv"
sample36_ids_csv = OUT_DIR / "feina_repr30_test_sample36_representative_ids.csv"

sample36_df.to_csv(sample36_csv, index=False, encoding="utf-8-sig")
sample36_df[[base_id_col]].rename(columns={base_id_col: "row_id"}).to_csv(
    sample36_ids_csv, index=False, encoding="utf-8-sig"
)

print("\nGuardado sample36:", sample36_csv)
print("Guardado ids      :", sample36_ids_csv)

```

Guardado sample36: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/final_sample_36/feina_repr30_test_sample36_representative.csv
Guardado ids : /home/harielpadillasanchez/Documentos/TT/TT2/outputs/final_sample_36/feina_repr30_test_sample36_representative_ids.csv

```
[30]: # =====
# 9) resumen
# =====
print("\nResumen")
print("-" * 60)
print("Test base:", len(df_feat))
print("Muestra final:", len(sample36_df))
print("Estratos en test:", df_feat[STRATUM_COL].nunique())
print("Estratos en muestra:", sample36_df[STRATUM_COL].nunique())

print("\nPrimeros row_id seleccionados:")
print(sample36_df[base_id_col].tolist()[:20])

display(
    sample36_df[STRATUM_COL]
    .value_counts()
    .rename_axis(STRATUM_COL)
    .reset_index(name="count")
    .head(20)
)
```

Resumen

Test base: 238

Muestra final: 36

Estratos en test: 39

Estratos en muestra: 35

Primeros row_id seleccionados:

[550, 571, 885, 1193, 1391, 1399, 1636, 1740, 1742, 1790, 1920, 2027, 2065, 2365, 2449, 2550, 2657, 2703, 2732, 2947]

	split_stratum	count
0	fam=reduction rules=r2 len=medium	2
1	fam=reduction rules=r2 len=short	1
2	fam=structural rules=r3 len=short	1
3	fam=structural rules=r3 len=medium	1
4	fam=lexical rules=r2 len=short	1
5	fam=lexical rules=r3 len=long	1
6	fam=structural rules=r2 len=short	1
7	fam=lexical rules=r1 len=long	1
8	fam=lexical rules=r1 len=medium	1
9	fam=unknown rules=r1 len=short	1

10	fam=structural rules=r1 len=medium	1
11	fam=reduction rules=r1 len=long	1
12	fam=structural rules=r1 len=short	1
13	fam=morphosyntactic rules=r1 len=medium	1
14	fam=reduction rules=r1 len=short	1
15	fam=lexical rules=r1 len=short	1
16	fam=lexical rules=r3 len=short	1
17	fam=morphosyntactic rules=r4plus len=long	1
18	fam=lexical rules=r2 len=long	1
19	fam=structural rules=r4plus len=long	1